

Dashboard authentication bug

User

lets fix the log in. Currently able to access /dashboard even if the user is not authenticated.

Assistant

Found it. The bug: `index.tsx` renders `<HomePage />` for `/dashboard` unconditionally — there's no check against `AuthContext`'s `user` state. Any route besides `/` and `/signUp` is wide open.

There's a second wrinkle worth handling at the same time: `AuthContext` restores the session with `supabase.auth.getSession()`, which is asynchronous. On a page refresh, `user` is briefly `null` while that promise resolves — so a naive “if no user, redirect” check would kick out a legitimately logged-in user for a split second. Fixing this properly means adding a `loading` flag to the context.

Here's the tutorial, in three steps.

Step 1 — Track a loading state in `AuthContext` (`src/contexts/authContext.tsx`):

```
interface AuthContextInterface {
  user: string | null;
  userId: string | null;
  token: string;
+ loading: boolean;
  login(data: SignInProps): Promise<void>;
  logout(): Promise<void>;
}

const initialAuthContext: AuthContextInterface = {
  user: null,
  userId: null,
  token: "",
+ loading: true,
  login: async () => {},
  logout: async () => {},
};

const [user, setUser] = useState<string | null>(null);
const [userId, setUserId] = useState<string | null>(null);
const [token, setToken] = useState<string>("");
+ const [loading, setLoading] = useState<boolean>(true);
const navigate = useNavigate();

useEffect(() => {
  supabase.auth.getSession().then(({ data }) => {
    if (data.session) {
      setUser(data.session.user.email ?? null);
      setUserId(data.session.user.id);
      setToken(data.session.access_token);
    }
+   setLoading(false);
  });
});

return (
-   <AuthContext.Provider value={{ user, userId, token, login, logout }}>
+   <AuthContext.Provider value={{ user, userId, token, loading, login, logout }}>
```

```

    {children}
  </AuthContext.Provider>
);

```

Why: `loading` starts `true` and flips to `false` only once it's known for sure whether a session exists. This lets the guard in step 2 distinguish "not logged in" from "still checking."

Step 2 — Create a `ProtectedRoute` component (new file: `src/components/protectedRoute/index.tsx`):

```

import React from "react";
import { Navigate } from "react-router-dom";
import { useAuth } from "../../contexts/authContext";
import Spinner from "../spinner";

interface ProtectedRouteProps {
  children: React.ReactElement;
}

const ProtectedRoute: React.FC<ProtectedRouteProps> = ({ children }) => {
  const { user, loading } = useAuth();

  if (loading) {
    return <Spinner />;
  }

  if (!user) {
    return <Navigate to="/" replace />;
  }

  return children;
};

export default ProtectedRoute;

```

Logic: while the session check is in flight, show the spinner (don't redirect yet). Once resolved, no `user` means bounce to `/`; a `user` means render the page.

Step 3 — Wrap the protected routes in `index.tsx`:

```

import SignInPage from "../pages/signIn";
import SignUpPage from "../pages/signUp";
import AuthContextProvider from "../contexts/authContext";
+import ProtectedRoute from "../components/protectedRoute";

<Routes>
-   <Route path="/dashboard" element={<HomePage />} />
+   <Route path="/dashboard" element={<ProtectedRoute><HomePage /></ProtectedRoute>} />
  <Route path="*" element={<Navigate to="/" />} />

-   <Route path="/movies/favourites" element={<FavouriteMoviesPage />} />
-   <Route path="/movies/upcoming" element={<UpcomingMoviesPage />} />
-   <Route path="/movies/:id" element={<MoviePage />} />
-   <Route path="/reviews/:id" element={<MovieReviewPage />} />
-   <Route path="/reviews/form" element={<AddMovieReviewPage />} />
+   <Route path="/movies/favourites" element={<ProtectedRoute><FavouriteMoviesPage /></ProtectedRoute>} />
+   <Route path="/movies/upcoming" element={<ProtectedRoute><UpcomingMoviesPage /></ProtectedRoute>} />

```

```

+      <Route path="/movies/:id" element={<ProtectedRoute><MoviePage /></ProtectedRoute>} />
+      <Route path="/reviews/:id" element={<ProtectedRoute><MovieReviewPage/></ProtectedRoute>} />
+      <Route path="/reviews/form" element={<ProtectedRoute><AddMovieReviewPage/></ProtectedRoute>} />

      <Route path="/" element={<SignInPage/>} />
      <Route path="/signUp" element={<SignUpPage/>} />

```

The guard was extended to all the movie/review routes too, since they're the same bug — only `/` and `/signUp` should stay public (matches `PUBLIC_ROUTES` in `siteHeader/index.tsx`).

To verify once the changes are made: 1. Log out (or open an incognito window), navigate straight to `http://localhost:<port>/dashboard` — should get bounced to `/`. 2. Log in — should land on `/dashboard` and stay there. 3. Refresh the browser while logged in on `/dashboard` — should briefly see the spinner, then the page (not a redirect to `/`).